



GROUP ASSIGNMENT (PART 2)

042024-MAL

Programming with Python

Intake code: UCDF2307ICT(SE)

Subject: Programming with Python

Title of the Assignment: Inventory Management System for PPE – Group 28

Due Date: 2nd August 2024

Name of group members:

Name	TP Number
Jayden Foo	TP073590
Elvan Sea Meng Jin	TP075849
Lai Xiao Chun	TP076509
Kee Genni	TP075968

Table of Contents

1.0 Program Source Code with Explanation.....	4
1.1 Import Modules and Assigning Variables	4
1.2 Main Function.....	5
1.3 Sign Up Function.....	6
1.4 Login Function.....	7
1.5 Read, Write, Append Files Function.....	10
1.6 PPE File Function	11
1.7 Supplier File Function	13
1.8 Hospital File Function	15
1.9 Distribution File Function	17
1.10 Distribution Hospital Function	20
1.11 Update Inventory Function	22
1.12 Distribute Inventory Function	24
1.13 Update Supplier Function	25
1.14 Update Hospital Function.....	28
1.15 Track Items Function	30
1.16 Search Distribution Function	31
1.17 Suppliers with PPE Function.....	32
1.18 Overall Transaction Function	34
1.19 Home Function & Code	36
2.0 Sample Input/Output with Explanation	39
2.1 Logging In.....	39
2.2 Creating Supplier File.....	40
2.3 PPE File Creation	41
2.4 Home Page - Option 1	42
2.5 Home Page - Option 2	43
2.6 Home Page - Option 3	44
2.7 Home Page - Option 4	45

2.8 Home Page - Option 5	46
2.9 Home Page - Option 8	47
3.0 Screenshots of Text Files	48
4.0 Conclusion	50
5.0 Flowchart-New	

1.0 Program Source Code with Explanation

1.1 Import Modules and Assigning Variables

```
✓ from datetime import datetime
import os
import re

ppe_path = "ppe.txt"
supp_path = "supplier.txt"
hosp_path = "hospital.txt"
dist_path = "distribution.txt"
receive_path = "receive_supply.txt"
user_info_path = 'user_info.txt'
```

Before beginning the code, three modules are imported into our code, which are the 'datetime', 'os' and 're' modules. The datetime module allows us to fetch the current date and time, while also performing arithmetic operations with it. Meanwhile, the 'os' module allows us to interact with the operating system to perform tasks such as checking paths, manipulating dictionaries and process control. And lastly, the 're' module is used for regular expressions.

After that, variables such as ppe_path, supp_path, hosp_path, dist_path, receive_path and user_info_path are set to their respective text files ("ppe.txt", "supplier.txt" etc.) to ease the calling process later.

1.2 Main Function

```
# Main
def main():
    if os.path.exists(user_info_path):
        users = load_users(user_info_path)
        user_count = len(users)
    else:
        users = {}
        user_count = 0

    attempts = {}
    while True:
        user_input = str(input("Would you like to sign up or log in?\n1. Sign up\n2. Log in\n3. Quit Program"))
        if user_input == "1":
            users = load_users(user_info_path)
            user_count = len(users)
            user_count, sign_up_success = sign_up(users, user_count, user_info_path)
            if sign_up_success:
                home()
        elif user_input == "2":
            users = load_users(user_info_path)
            if login(users, user_info_path, attempts) == True:
                home()
                print("this is home")
        elif user_input == "3":
            print("Quitting program")
            return
        else:
            print("Please enter correct input")
```

Next, a function main() is defined. This function is the main part and beginning of the code, where users will run the code. The program checks whether the path “user_info_path” which is defined earlier exists or not. If so, the program will check for the number of users which are already registered in the system by checking the text file linked to the path. If “user_info_path” does not exist, the number of users in the system will be set to 0. After that, in a while True loop, the program will prompt the user to sign up, login or quit the program.

If the user chooses to sign up, the program will retrieve existing user details (if it exists) and call the sign_up function. And directs the user to the home page if successful.

If the user chooses to log in, the program retrieves the existing user details (if it exists) and calls the login function and directs the user to the home page if successful.

If the user chooses to quit the program, the program will end.

However, if the user chooses none of the above options, the program will print an error and repeat the entire process.

1.3 Sign Up Function

```
# Sign Up
def sign_up(users, user_count, user_info_path):
    #Check if user count more than 4
    while True:
        if user_count >= 4:
            print("The maximum number of registered users has been reached.")
            return user_count, False #return exit whole function
        else:
            username = input("Please enter a username")
            if username in users:
                print("Username already exists. Please choose a different username.")
            else:
                password = input("Please enter your password")
                users[username] = password
                print(f"Your username is {username} \nYour password is {password}")
                print("Sign up successfully")

                user_count+=1
                content = f"{username}, {password}"
                append_file(user_info_path, content)

                print("You have logged in")
                print("Welcome to the home page")
                return user_count, True
```

The `sign_up` function begins by requesting the parameters “users”, “user_count” and “user_info_path”. Next, it checks whether the number of users has reached the maximum number allowed, which is 4. If so, the program prints an error and exits this function.

If the number of users is below 4, it will prompt for a username and check whether it exists in “users”. Assuming it does not exist, the user will need to enter a password to successfully register. After that, the `user_count` variable increases by one and the user details are appended to the text file.

The user will be redirected to the home page if the sign-up is successful.

1.4 Login Function

```
def load_users(user_info_path):  
    users = {}  
    try:  
        user_content = readlines_file(user_info_path)  
        for line in user_content:  
            username, password = line.strip().split(' ')  
            users[username] = password  
    except FileNotFoundError:  
        print(f"There are no users registered. Please sign up before logging in.")  
    return users
```

The login function begins by defining a function called “load_users” to load the user details from the text file. The program will remove the empty spaces and split the username and password inside the text file and assign them to their respective variables, “username” and “password”. However, if the text file is not found, the program prints that there are no users registered and asks the user to sign up before logging in.

```
def update_users_info(user_info_path, users):  
    content = []  
    for username, password in users.items():  
        content.append(f"{username}, {password}\n")  
    write_file(user_info_path, content)  
  
def delete_user(user_info_path, username_to_delete):  
    users = load_users(user_info_path)  
    if username_to_delete in users:  
        del users[username_to_delete]  
        update_users_info(user_info_path, users)
```

Next, two functions are defined together, which is the “update_users_info” and “delete_user” function. The first function appends the username and password into an empty list. The second function loads user details from the text file and deletes them if the username is in the text file. Overall, both functions are to be called when user login attempts exceed 3 times, in which the user details will be deleted from the database.

```
def login(users, user_info_path, attempts, max_attempts=3):
    if not os.path.exists(user_info_path):
        print(f"There are no users registered. Please sign up before logging in.")
        return

    users = load_users(user_info_path)
    if not users:
        return

    while True:
        username = input("Please enter your username: ")

        # If username is not valid
        if username not in users:
            print("This username is not found. Please try again.")
            continue

        password = input("Please enter your password: ")

        # If attempts more than 3 times
        if username in attempts and attempts[username] >= max_attempts:
            print("Too many failed attempts. Access terminated.")
            delete_user(user_info_path, username)
            return False

        # If login successful
        if users[username] == password:
            print("Login successful!")
            return True
        else:
            if username in attempts:
                attempts[username] += 1
            else:
                attempts[username] = 1

            if attempts[username] >= max_attempts:
                print("You have been banned from too many attempts.")
                delete_user(user_info_path, username)
                return False
            else:
                print("Login failed. Please try again.")
```


Finally, the main login function, `login()` is defined. It starts by checking whether the text file path exists or not. If it does not exist, the program exists from the login function as there are no users registered.

If it exists, the program loads the user details from the text file into a variable named “users”. It then prompts users for their username. If the username is not in “users”, it prints an error and one count is added to failed username attempts, while asking the user to retry. If the username is found, it asks for the password.

If the failed username attempts exceed the max attempts limit, the user is terminated from the program.

However, if the failed username attempts are below three times, the program then verifies the password of the specific user. If it matches, the program will log the user in and skip this function. If it does not match, it checks whether the amount of login attempts from this user exceeds the limit and terminates their access if so. If user access is terminated, their account details are removed from the database.

1.5 Read, Write, Append Files Function

```
# RWA File
def read_file(file_path):
    with open(file_path, 'r') as file:
        return file.read()

def readlines_file(file_path):
    with open(file_path, 'r') as file:
        return file.readlines()

def write_file(file_path, content):
    with open(file_path, 'w') as file:
        file.writelines(content)

def append_file(file_path, content):
    with open(file_path, 'a') as file:
        file.writelines(content + '\n')
```

The above functions are used to simplify the process of opening files.

The function “read_file(file_path)” opens a file in read mode and read the first line.

The “readlines_file” function opens a file in read mode and read every line of the file.

The “write_file” opens a file in write mode, or “w”, and writes a content inside the file.

The “append_file” function opens a file in append mode, or “a”, and append lines to the bottom of files.

1.6 PPE File Function

```

# ppe_file
def create_ppe_file(ppe_path, supp_path):

    # Check if the file already exists
    if os.path.exists(ppe_path):
        return

    else:
        print(f"{ppe_path} does not exist. Creating a new file.")

    # Open the file in write mode
    print(f"Example: HC, Head Cover, 100, S01, January 27 2024 \n(The user input should be exactly the same as the expected format)")

    while True:

        # Prompt user to enter text or 'done' to finish
        # Expected input
        # HC, Head Cover, 100, S01, January 27 2024

        user_input = input("Enter text to write into this file or 'done' to finish:")

        if user_input.lower() == 'done':
            break

        try:
            line = user_input.strip().split(',')
            print(line)

            if len(line) != 5:
                print("Your input does not contain the expected number of elements.")
                continue

            # If the user input matches the format
            ppe_code = re.match( pattern: r'^[A-Z]{2}$', line[0]) is not None
            ppe_name = re.match( pattern: r'^[A-Za-z ]+$', line[1]) is not None
            ppe_quantity = line[2].isdigit() and int(line[2]) == 100

            supp_content = read_file(supp_path)
            ppe_supplier = line[3] in supp_content

            date_format = "%B %d %Y"
            ppe_date = False

            try:
                # If the text is in the specified (correct) date format
                parsed_date = datetime.strptime(line[4], date_format)
                ppe_date = True

            except ValueError:
                # If the text is not in the specified date format
                ppe_date = False

            if ppe_code and ppe_name and ppe_quantity and ppe_supplier and ppe_date:
                write_file(ppe_path, user_input)
                print("Data written to file.")

            else:
                print("Input data is not valid based on the criteria.")
                if not ppe_code:
                    print("Invalid PPE code.")
                if not ppe_name:
                    print("Invalid PPE name.")
                if not ppe_quantity:
                    print("Invalid PPE quantity.")
                if not ppe_supplier:
                    print("Supplier not found.")
                if not ppe_date:
                    print("Invalid date format.")

            except Exception as e:
                print(f"An error occurred: {e}")

        print(f"{ppe_path} has been created with your input")

```

First, the program defines the “create_ppe_file” function and begins by checking whether the ppe_path exists or not. If it exists, the function ends (which represents that the program has been run before). If it does not exist, a new file is created and opened in write mode (which occurs when the program is run for the first time). It then prompts the user for input and provides a format for data to be inputted in. Once the user is done inputting data (according to the specific format), they can input “done” to end the prompt. Content inside ppe_file can only be “write” into at this beginning stage, users wont be be able to add values into ppe_file anymore after input “done”.

The program then checks the input by removing the empty spaces and splitting them into multiple parts and checks whether each line has exactly 5 elements. If not, it proceeds to check the errors one by one. The “ppe_code”, “ppe_name” and “ppe_quantity” is checked to match a predefined format. For example, the “ppe_code” is checked whether it starts with two capital letters, the “ppe_name” is checked for whether it starts with a capital letter and ends in non-capital letters and the “ppe_quantity” must be a digit ending in 100. The “supp_path” file is then set into read mode under the “supp_content” variable. The “ppe_supplier” variable is set to the fourth line in “supp_content”, which verifies the existence of the supplier code. After that, the date format is set to %B \$d \$Y which represents month, day and year.

Once all the variables are valid and following the correct format, all of them will be inserted into the file. If any of them does not follow the criteria, the respective variables will be highlighted and printed for the user to correct.

1.7 Supplier File Function

```

# supplier_file
def supplier_file(supp_path):
    # Check if the supplier file exists
    if not os.path.exists(supp_path):
        print(f"{supp_path} does not exist. Creating a new file.")

    else:
        # print("File exists")
        return

    supply_count = 0 # Initialize the supply count

    print("Example: S01, Supplier Name, 1 Jalan XXX Taman XXX, 012-3456789 \n(The user input should be exactly the same as the expected format)")
    print("Noted: The address should not include any ', ' ")

    while True:
        # Prompt user for supplier details
        # Expected input
        # S01, Supplier Name, 8 Jalan XXX, 012-3456789
        user_input = input("Please enter supplier details or type 'done' to finish: ")

        # Check if user is done or supply count is 4 or more
        if user_input.lower() == 'done' or supply_count >= 4:
            break

        try:
            line = user_input.strip().split(' ')
            print(line)

            if len(line) != 4:
                print("Your input does not contain the expected number of elements.")
                continue

            supp_code = re.match(pattern: r'^S\d{2}$', line[0]) is not None
            supp_name = re.match(pattern: r'^[A-Z][A-Za-z ]+$', line[1]) is not None
            supp_address = re.match(pattern: r'^[A-Za-z0-9/ ]+$', line[2]) is not None
            supp_contact = re.match(pattern: r'^\d{3}-\d{7,8}$', line[3]) is not None

            if supp_code and supp_name and supp_address and supp_contact:
                append_file(supp_path, user_input)
                print("Data written to file.")

            else:
                print("Input data is not valid based on the criteria.")
                if not supp_code:
                    print("Invalid supplier code.")
                if not supp_name:
                    print("Invalid supplier name.")
                if not supp_address:
                    print("Invalid supplier address.")
                if not supp_contact:
                    print("Invalid supplier contact.")

            supply_count += 1
            if supply_count >= 4:
                print("The supplier count has reached the maximum.")
                break

        except Exception as e:
            print(f"An error occurred: {e}")

    print(f"{supp_path} has been created with your input.")

```

Similarly to the ppe file function, the supplier file function begins by checking whether the `supplier_path` exists or not. If it does not exist, a new file will be created, or skipped if it exists.

The variable `supply_count` variable is initialized as 0, which will keep track of the number of suppliers later on. Next, the program prompts the user to input the supplier details following a format. Once the user is done or the `supply_count` is more than or equals to 4, the data inserted will be broken down and their formatting will be checked. The `supp_code` variable will be checked whether it starts with S followed by two digits. The `supp_name` variable will be checked whether it starts with a capital letter, followed by non-capital letters. The `supp_address` can only have numbers and letters. The `supp_contact` will be checked whether it starts with 3 numbers, followed by a hyphen, then followed by 7-8 digits.

Once all the details are confirmed to be following the format, all details will be appended into the supplier text file. If any of them are not following the format, they will be broadcasted to the user and asked to reinput the details. Each line of detail being inserted will add one to the `supply_count` and when it exceeds 4, the function will end.

1.8 Hospital File Function

```

# hospital_file
def hospital_file(hosp_path):

    if not os.path.exists(hosp_path):

        while True:
            user_input = str(input("Would you like to create hospital.txt file?\n '1' for Yes\n '2' for No"))

            if user_input == "1":
                hosp_count = 0
                print("Example: H01, Hospital XXX")

                while True:
                    #Expected input
                    #H01, Hospital Name XXX
                    hosp_details = input("Please enter hospital details or type 'done' to finish: ")

                    if hosp_details.lower() == "done" or hosp_count >= 4:
                        break

                    try:
                        line = hosp_details.strip().split(', ')
                        print(line)

                        if len(line) != 2:
                            print("Your input does not contain the expected number of elements.")
                            continue

                        hosp_code = re.match(pattern=r'^H\d{2}$', line[0]) is not None
                        hosp_name = re.match(pattern=r'^[A-Z][A-Za-z ]+$', line[1]) is not None

                        if hosp_code and hosp_name:
                            append_file(hosp_path, hosp_details)
                            hosp_count += 1
                            print("Data written to file.")

                        else:
                            print("Input data is not valid based on the criteria.")
                            if not hosp_code:
                                print("Invalid hospital code.")
                            if not hosp_name:
                                print("Invalid hospital name")

                    except Exception as e:
                        print(f"An error occurred: {e}")

                    if hosp_count >= 4:
                        print("The hospital count has reached the maximum number.")
                        break

                print(f"{hosp_path} has been created with your input.")
                break #Break after user done creating and entering the file

            elif user_input == '2':
                break

            else:
                print("Please enter the correct input")

    but(("pozbitsef'fxf oflesqA exfzfe' kefncufud fo pomebade,")
    efze:

```

The program defines a function “hospital_file”. First, it checks whether the hospital path, or hosp_path exists or not. If it already exists, the user will be notified and sent back to the home page. If it does not exist, the user will be asked whether they would like to create a hospital file. If the user wants to create a hospital file, the hosp_count which keeps track of the number of hospitals is set to 0. The user is then prompted to input hospital details. Once the user is done or the number of hosp_count is more than or equals to 4, the prompt will end. The inputted hospital data will have its empty spaces removed and split and the format is checked. If the length of the lines is not equal to 2, the program will analyze the error by checking the format of the hospital code and name.

If both data inserted is correct, the hosp_name and hosp_code will be appended into the hospital text file and the hosp_count will be increased by 1.

If either one of the data produces an error, that specific data will be highlighted and displayed to the user to be reinputted.

1.9 Distribution File Function

```

#distribution_file
!usage
def distribution_file(dist_path, ppe_path, hosp_path):
    while True:
        item_code = input("What item do you want to distribute? (please type in Item Code): ").upper()

        lines = readlines_file(ppe_path)

        # Check if user input is a valid item_code
        item_exists = False
        for line in lines:
            item = line.strip().split(", ")
            if item[0] == item_code:
                item_exists = True
                break

        if not item_exists:
            print("This item does not exist. Please enter a valid item code")
            continue

        while True:
            amount = input("Please input the amount of items to be distributed: ")
            try:
                amount = int(amount)
                updated_lines = []
                quantity_sufficient = True

                for line in lines:
                    item = line.strip().split(", ")

                    if item[0] == item_code:          # HC, Head Cover, 100, S01, DATE
                        current_quantity = int(item[2])

                        # Check if item quantity is sufficient
                        if amount > current_quantity:
                            print("There is not enough of this item for distribution.")
                            quantity_sufficient = False
                            break # Exit for loop

                        while True:
                            hosp_code = input("Which hospital do you want to distribute to? (please type in Hospital Code): ").upper()
                            hosp_content = read_file(hosp_path)

                            if hosp_code not in hosp_content:
                                print("This hospital does not exist. Please enter a valid code.")
                            else:
                                new_quantity = current_quantity - amount
                                item[2] = str(new_quantity)
                                current_date = datetime.datetime.now()
                                date_formalized = current_date.strftime("%B %d %Y")
                                content = f"{item_code}, {amount}, {hosp_code}, {date_formalized}"
                                append_file(dist_path, content)

                                print("Your distribution has been recorded successfully.")
                                break # Exit hosp_code loop

                        updated_lines.append(", ".join(item) + "\n")
                    else:
                        updated_lines.append(line)

                if quantity_sufficient:
                    write_file(ppe_path, updated_lines)
                    break # Exit item code loop
            except ValueError:
                print("This is not a valid number.")

        # Ask if the user wants to continue
        while True:
            choice = input("Do you want to continue distributing items? (yes/no): ").lower()
            if choice in ('yes', 'no'):
                break # Exit choice and continue with distribution
            else:
                print("Invalid choice. Please enter 'yes' or 'no'.")

        if choice == 'no':
            break # Exit distribution

```

The function `distribution_file()` has been defined. This function handles the distribution of PPE items to various hospitals. It has three parameters: `dist_path`, `ppe_path`, and `hosp_path`.

The function enters an infinite loop, prompting the user to enter the item code of the PPE they want to distribute. What the user enters is stored in the variable `item_code`.

If `item_exists` is `False`, meaning that the item code does not exist in the inventory, an error message is returned. The user is prompted to re-enter the item code and continue the outer loop.

If `item_exists` is `True`, the function enters another loop, prompting the user to enter the quantity of the item to be distributed. The input is stored in the variable `amount`. The program attempts to convert `amount` to an integer. If the conversion fails, an error message is displayed. The user is prompted to enter the amount again.

If the conversion is successful, the program initializes the empty list `updated_lines` and the boolean variable `quantity_sufficient` to `True`. Then, if the first element of `item` matches the `item_code` (which means the item code based on user input is found), the current quantity of the item is stored in `current_quantity`.

The program checks if the amount to be distributed is greater than the `current_quantity`. If yes, an error message is displayed, and `quantity_sufficient` is set to `False`. The loop breaks.

If not, the `quantity_sufficient` will set `True`, the program enters another loop, prompting the user to enter a hospital code. The input is converted to upper case and stored in the variable `hosp_code`. The hospital file is read using `read_file(hosp_path)` and the contents are stored in `hosp_content`.

The program checks if `hosp_code` exists in `hosp_content`. If not, an error message is displayed (which means the hospital code which input from user is not valid), and the user is prompted to enter the hospital code again.

If it exists, the new quantity of the item is calculated by subtracting from `hosp_code`, and the new quantity is updated in the list. The current date and time are obtained using `datetime` function and the date is formatted to “%B %d %Y”. A distribution record is created using `,` and it is appended to the distribution file using `.` A success message is displayed, and the loop breaks.

The updated item details are appended to the `updated_lines` list. If the `quantity_sufficient` flag is `True`, the updated inventory is written back to the PPE file using `write_file(ppe_path, updated_lines)`. The loop for the item code will break.

The program asks the user if they want to continue distributing items. The input is stored in the variable `choice`. If the user enters "yes" or "no", the loop will break. Otherwise, an error message is displayed, and the user is prompted to enter "yes" or "no" again.

If the user chooses 'no', the outer loop will break, and the function ends.

1.10 Distribution Hospital Function

```
# distribution_hospital
def distribution_hospital():
    hospitals = {}
    distribution = {}

    hosp_content = readlines_file(hosp_path)

    for line in hosp_content:
        item = line.strip().split(' ')
        if len(item) == 2:
            hosp_code, hosp_name = item
            hospitals[hosp_code] = hosp_name

    dist_content = readlines_file(dist_path)

    for line in dist_content:
        item = line.strip().split(' ')
        if len(item) >= 3:
            item_code = item[0]
            quantity = int(item[1])
            hosp_code = item[2]

            # Initialize dictionary for the hospital if it doesn't exist
            if hosp_code not in distribution:
                distribution[hosp_code] = {}

            # Sum up quantity for the same product and hospital
            if item_code in distribution[hosp_code]:
                distribution[hosp_code][item_code] += quantity
            else:
                distribution[hosp_code][item_code] = quantity

    # Print the distribution summary
    for hosp_code, items in distribution.items():
        for item_code, total_quantity in items.items():
            hosp_name = hospitals.get(hosp_code, "Unknown Hospital")
            print(f"{hosp_name} ({hosp_code}) has been distributed {total_quantity} of {item_code}")
```

The `distribution_hospital()` function is defined. Two variables, “hospitals” and “distribution” is set as empty dictionaries. The hospital path text file and distionary path text file is set to read mode. The program then removes empty spaces and splits the data in `hosp_content`, saved in a variable called “item” and checks their length. If the length equals to 2, `hosp_code` and `hosp_name` is set as “item”.

The program removes empty spaces and splits each line in `dist_content` and checks for its length. If the length is more than or equal to 3, each split element in `dist_content` is respectively set to “item_code”, “quantity” and “hosp_code” variables.

Next, the program initializes a dictionary for the hospital if it does not exist.

After that, it sums up the quantity for each product in each hospital and prints them in a report form.

1.11 Update Inventory Function

```
# update_inventory
def add_invent(ppe_path, receive_path):
    while True:
        item_code = input("What item do you want to add? (Please enter item code): ")
        ppe_content = readlines_file(ppe_path)

        # Check if item exists in PPE (is it a VALID item?)
        if not any(item_code in line for line in ppe_content):
            print("This item does not exist. Please enter a valid code")

        else:
            try:
                amount = int(input("How many items to add?: "))
                if amount <= 0:
                    print("Amount must be a positive integer greater than zero.")
                    continue

            except ValueError:
                print("This is not a valid number")

            else:
                update_cont = []

                for line in ppe_content:
                    parts = line.strip().split(', ')
```

To add a certain number of things to an inventory, The `add_invent(ppe_path, receive_path)` function is used to add a certain quantity of items to the inventory. The function will continuously ask the user to enter an item code (while True) at the beginning. After that, it will see if the file at `ppe_path` has this item code. If the item code is not found, it prompts the user to enter a valid code. If the item code is found and valid, it then prompts the user to enter a positive integer number of items to add. If the input for the amount is invalid, it prompts the user to input again. After valid inputs are received, each line in the PPE file is split into parts.

```
# HC, Head Cover, 100, S01, January 27 2024
if len(parts) >= 5 and parts[0] == item_code:
    curr_quantity = int(parts[2])
    new_quantity = curr_quantity + amount
    current_date = datetime.datetime.now()
    date_formalized = current_date.strftime("%B %d %Y")

    parts[2] = str(new_quantity)
    supp_code = parts[3]
    parts[4] = date_formalized
    updated_line = ', '.join(parts) + '\n'
    update_cont.append(updated_line)

else:
    update_cont.append(line)

write_file(ppe_path, update_cont)
receive_content = f"{item_code}, {amount}, {supp_code}, {date_formalized}"
append_file(receive_path, receive_content)

print("Inventory updated successfully.")
return_# Quit function after successful update
```

After ppe_content is split successfully, the function checks if the length of the input line has at least five parts and check if the first part is the same as the item code that user input. If yes, the function adds the specified amount of the item and sets the current date. The updated line is then appended to the list of updated_line, while unchanged lines are also appended into this list. If the length of parts is not equal to 5 or the first part is not the same as the item code that user input, the function appends the unchanged line to the update_cont list.

After processing of all lines is completed, the function writes the updated lines back to the PPE file, logs the transaction details (item code, amount, supplier code, date) to the reception file. Then, a message is printed informing the user of the success of updating inventory. Finally, the function will be exited.

1.12 Distribute Inventory Function

```
# distribute_inventory
def distribute_invent(hosp_path, dist_path, ppe_path):
    # Check if paths exist
    if os.path.exists(hosp_path):
        distribution_file(dist_path, ppe_path, hosp_path)

    else:
        print("Please create hospital file before distributing inventory")
        # Create hosp file if not exists
        hospital_file(hosp_path)
```

The `distribute_inventory` function starts by checking whether the `hosp_path` variable exists. If it does not exist, the user will be asked to create a hospital file before distributing the inventory, in which they will be linked to the `hospital_file` function. If it exists, the function “`distribution_file`” will be called.

1.13 Update Supplier Function

```

# update_supplier
def update_supp_details(supp_path):
    supp_content = readlines_file(supp_path)
    while True:
        supp_code = input("Please enter the supplier code: ")

        #Check if is a valid supplier code
        if not any(supp_code in line for line in supp_content):
            print("This supplier code does not exist. Please enter a valid code")
            continue

        while True:
            details = input("Which detail do you want to edit? [Name, Address, Contact]: ").lower()
            updated_content = []

            if details == "name":
                while True:
                    new_name = input("Please enter the new name: ")

                    if re.match(pattern: r'^[A-Z][A-Za-z ]+$' , new_name):
                        # Update supplier name
                        for line in supp_content:
                            if supp_code in line:
                                parts = line.strip().split(' ')
                                parts[1] = new_name
                                updated_line = ' '.join(parts) + '\n'
                                updated_content.append(updated_line)

                            else:
                                updated_content.append(line)

                        write_file(supp_path, updated_content)
                        print("Supplier name updated successfully.")
                        return

                    else:
                        print("Invalid name format. Please enter alphabetical characters only starting with uppercase.")
                        print("Example: Supplier Name XXX")

            elif details == "address":
                while True:
                    new_address = input("Please enter the new address: ")

                    if re.match(pattern: r'^[A-Za-z0-9/ ]+$' , new_address):
                        # Update supplier address
                        for line in supp_content:
                            if supp_code in line:
                                parts = line.strip().split(' ')
                                parts[2] = new_address
                                updated_line = ' '.join(parts) + '\n'
                                updated_content.append(updated_line)

                            else:
                                updated_content.append(line)

```

```

# update_supplier
def update_supp_details(supp_path):
    supp_content = readlines_file(supp_path)
    while True:
        supp_code = input("Please enter the supplier code: ")

        #Check if is a valid supplier code
        if not any(supp_code in line for line in supp_content):
            print("This supplier code does not exist. Please enter a valid code")
            continue

        while True:
            details = input("Which detail do you want to edit? [Name, Address, Contact]: ").lower()
            updated_content = []

            if details == "name":
                while True:
                    new_name = input("Please enter the new name: ")

                    if re.match(pattern: r'^[A-Z][A-Za-z ]+$' , new_name):
                        # Update supplier name
                        for line in supp_content:
                            if supp_code in line:
                                parts = line.strip().split(' ')
                                parts[1] = new_name
                                updated_line = ', '.join(parts) + '\n'
                                updated_content.append(updated_line)

                            else:
                                updated_content.append(line)

                        write_file(supp_path, updated_content)
                        print("Supplier name updated successfully.")
                        return

                    else:
                        print("Invalid name format. Please enter alphabetical characters only starting with uppercase.")
                        print("Example: Supplier Name XXX")

            elif details == "address":
                while True:
                    new_address = input("Please enter the new address: ")

                    if re.match(pattern: r'^[A-Za-z0-9/ ]+$' , new_address):
                        # Update supplier address
                        for line in supp_content:
                            if supp_code in line:
                                parts = line.strip().split(' ')
                                parts[2] = new_address
                                updated_line = ', '.join(parts) + '\n'
                                updated_content.append(updated_line)

                            else:
                                updated_content.append(line)

```

```

        write_file(supp_path, updated_content)
        print("Supplier address updated successfully.")
        return

    else:
        print("Invalid address format. Please enter alphanumerical characters '/' or spaces only.")
        print("Example: 1 Jalan 1/2 Taman XXX")

elif details == "contact":
    while True:
        new_contact = input("Please enter the new contact: ")

        if re.match(pattern=r'\d{3}-\d{7,8}$', new_contact):
            # Update supplier address
            for line in supp_content:
                if supp_code in line:
                    parts = line.strip().split(' ')
                    parts[3] = new_contact
                    updated_line = ', '.join(parts) + '\n'
                    updated_content.append(updated_line)

                else:
                    updated_content.append(line)

            write_file(supp_path, updated_content)
            print("Supplier address updated successfully.")
            return

        else:
            print("Invalid name format. Please enter 3 digits, followed by a '-', and end with either 7 or 8 digits.")
            print("Example: 012-3456789")

    else:
        print("Please enter the correct details.")

```

The `update\_supp\_details` function is used to update supplier information in the supplier file. First, it reads the file's content and will prompt the user to enter a supplier code. Then, the function will verify if the code exists in the file. If the code isn't found the function will prompt the user to enter a valid code. This process will be looped until a valid supplier code is input.

Once a valid code is entered, the function will proceed and asks the user which detail you want to edit name, address, or contact. For each choice, the function will execute a verification process for the new information the user provides, where all this happens in a loop. For example, if the user chooses to update the name, it ensures the new name starts with an uppercase letter and contains only letters and spaces. If you choose to update the address, it checks that it contains only letters, numbers, slashes, and spaces.

After that, if the new detail entered is valid, the function updates the relevant line in the file and saves the changes. Then, a message will be printed informing users that these details have been updated successfully.

1.14 Update Hospital Function

```

# update_hospital
def update_hosp_details(hosp_path):
    if os.path.exists(hosp_path):
        while True:
            hosp_code = input("Please enter the hospital code: ")
            hosp_content = readlines_file(hosp_path)

            # Check if hospitaSSL code is valid
            if not any(hosp_code in line for line in hosp_content):
                print("This hospital code does not exist. Please enter a valid hospital code.")
                continue

            else:
                while True:
                    details = input("Which detail do you want to edit? [Name]").lower()
                    if details == 'name':
                        new_name = input("Please enter the new name: ")

                        if re.match(pattern=r'^[A-Z][A-Za-z ]+$', new_name):
                            updated_content = []

                            for line in hosp_content:
                                # Update new hospital name
                                if hosp_code in line:
                                    parts = line.strip().split(' ')
                                    parts[1] = new_name
                                    updated_line = ', '.join(parts) + '\n'
                                    updated_content.append(updated_line)
                                else:
                                    updated_content.append(line)

                            write_file(hosp_path, updated_content)
                            print("Hospital name updated successfully.")
                            return

                        else:
                            print("Invalid name format. Please enter alphabetical characters only starting with an uppercase letter.")
                            continue

                    else:
                        print("Please enter a valid detail [Name].")
                        continue

            else:
                print("Please create hospital file before updating it.")
                hospital_file(hosp_path)
                return

```

The `update_hosp_details (hosp_path)` function is to update existing hospital details. To update hospital details, there must be at least one hospital file created. If there is none, a message “Please create hospital file before updating it” will be generated and redirect to the `hospital_file` function to create it.

If the hospital file exists, user will be first prompt to enter hospital code. Then, the function checks if the hospital code exists in the file. If it does not exist, it will print a message indicating that the input is invalid, and the user will be prompted again to enter a valid `hosp_code`. This loop will repeat until a valid `hosp_code` is given by the user. Once a valid code is entered, it prompts the user to specify which detail to edit, which for now, user can only edit hospital name. Then, they are to enter a new name, which must follow a certain format, such as the new name must start with an uppercase letter and contain only letters and spaces. If the new name does not match the required format or the input entered is not a “name”, the user is prompted to enter a valid name again and the loop will continue. If the new name matches this condition, the function prepares to update the hospital's details in the file.

Then, the function continues by executing a nested “if” statement. For each line in the hospital file, it checks if the line contains the hospital code. If yes, the line is split into parts where the name part is updated with the new name, and the new line is appended to the `updated_content` list.

If the line does not contain the hospital code, the functions append the line the `updated_content` list without any changes. After this, the function writes the updated content back to the file and prints a success message.

1.15 Track Items Function

```
# track_items
def track_items():
    ppe_content = readlines_file(ppe_path)
    ppe_content = [line.strip().split(' ') for line in ppe_content]
    while True:
        track = input("Do you want to track available quantity of all items or track all items which has less than 25 stock? [1 or 2]")

        # Print item sorted by item code in asc
        if track == '1':

            # Sort ppe_content by the first column (code)
            sorted_ppe = sorted(ppe_content, key=_lambda x: x[0])

            # Print Item Code and Quantity
            for line in sorted_ppe:
                print(f"Item Code: {line[0]}; Quantity: {line[2]}")

            break

        # Print item that less than 25
        elif track == '2':

            # Sort ppe_content by the third column (quantity with ASC)
            sorted_ppe = sorted(ppe_content, key=_lambda x: x[2])
            items_below_25 = False

            # Print Item and quantity(where less than 25)
            for line in sorted_ppe:
                if int(line[2]) < 25:
                    items_below_25 = True
                    break

            if items_below_25:
                print("Item belows 25 boxes:")
                for line in sorted_ppe:
                    if int(line[2]) < 25:
                        print(f"Item Code: {line[0]}; Quantity: {line[2]}")

            if not items_below_25:
                print(f"There are no items below 25 boxes.")

            break

        else:
            print("The input is invalid. Please try again.")
```

The function begins by reading all lines in the ppe_path file, removing empty spaces and splitting them. Then, it prompts the user to choose between tracking the available quantity of all items or tracking all items below 25 stocks. If the user chooses to track the available quantity of all items, the element in the file is sorted in ascending order by first element. After that, the program prints the sorted items by code and quantity.

If the user chooses to track all items below 25 stocks, the program sorts the elements in ascending order by quantity (which is the third element). Then, it filters out the items which quantity is below 25 and lists them down. If there are no items below 25 stocks, it will notify the user.

1.16 Search Distribution Function

```
# search_distribution
def search_distribution(ppe_path):
    while True:
        item_code = input("Please enter the Item Code")
        ppe_content = readlines_file(ppe_path)

        # Check if user input is valid item code
        if not any_(item_code in line for line in ppe_content):
            print('This input is invalid. Please try again.')

        else:
            dist_content = readlines_file(dist_path)

            # Check if user input has been distributed
            if not any_(item_code in line for line in dist_content):
                print(f"{item_code} does not distribute to any hospital yet.")
                return

            else:
                hosp_quantity = {} # Dict used to sum up the quantity for the same hosp/item
                for line in dist_content:
                    parts = line.strip().split(' ')
                    if parts[0] == item_code:
                        quantity = int(parts[1])
                        hosp_code = parts[2]

                        # Sum up quantity for the same hosp/item
                        if (item_code, hosp_code) in hosp_quantity:
                            hosp_quantity[(item_code, hosp_code)] += quantity
                        else:
                            hosp_quantity[(item_code, hosp_code)] = quantity

                for (item_code, hosp_code), quantity in hosp_quantity.items():
                    print(f"{item_code} has been distributed {quantity} to {hosp_code}")

                return
```

The function begins by prompting the user to input the item code and reading all lines in the ppe_path file. If the item code is not found in the file, an error will be printed. If the code is found, the program will read all lines in dist_path.

Next, it checks whether the item input by the user has been distributed to a hospital or not. If it has been distributed to a hospital, the items in the file will have empty spaces removed and split. If the items have been distributed to more than one hospital, the program sums up the items by adding

them together. If it has only been distributed to one hospital, the item quantity distributed will remain unchanged.

1.17 Suppliers with PPE Function

```
# suppliers_with_PPE
def supp_with_ppe(supp_path, ppe_path):
    suppliers = {}
    supp_content = readlines_file(supp_path)

    for line in supp_content:
        parts = line.strip().split(' ')
        suppliers[parts[0]] = parts[1]

    # print(suppliers)
    items = {}
    ppe_content = readlines_file(ppe_path)

    for line in ppe_content:
        parts = line.strip().split(' ')
        item_code = parts[0]
        item_name = parts[1]
        quantity = parts[2]
        supp_code = parts[3]

        if supp_code in suppliers: # Check if is valid supp_code
            if supp_code not in items: # Check whether the supp has alr supplied items b4
                items[supp_code] = []
            items[supp_code].append((item_code, item_name, quantity))

    for supp_code, item_list in items.items():
        print(f"Supplier: {suppliers[supp_code]}")
        for item in item_list:
            print(f"Item Code: {item[0]}, Item Name: {item[1]}, Quantity: {item[2]}")
        print(" ")
```

The function above initializes an empty dictionary “suppliers” and reads all lines in the file “supp_path”. Then, the lines in the file will remove all whitelines and are split. The supplier_code from the file is stored as the key and the supplier_name is then stored as the value in the “suppliers” dictionary.

Next, an empty dictionary “items” is initialized and all lines in “ppe_path” is read. The whitelines in the file is removed and split. The item_code, item_name, quantity and supp_code from the file will then be inserted into the dictionary “items” as slot 0, 1, 2, 3 respectively.

The supplier code will be validated by checking if it exists in “suppliers” dictionary. If it exists and is not already in “items” dictionary, a new list will be made for that supplier code. The item details for that supplier code will then be appended into the “items” dictionary.

Finally, all the details of suppliers and their item details are printed.

1.18 Overall Transaction Function

```

777 # overall_transactions
778 def overall_transaction(dist_path, receive_path):
779     valid_month = {
780         'January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September', 'October', 'November', 'December'
781     }
782     while True:
783         month = input("Please input the month which you would like to retrieve data from: [January, February...]).capitalize()
784
785         # If month not a valid month
786         if month not in valid_month:
787             print("Invalid month. Please enter a valid month name.")
788             continue
789
790         dist_content = readlines_file(dist_path)
791
792         # Transaction report for distribution
793         transactions_found = False
794         hosp_transaction = {}
795
796         for line in dist_content:
797             # If month found
798             if month in line:
799                 transactions_found = True
800                 parts = line.strip().split(',')
801                 hosp_code = parts[2]
802                 if hosp_code not in hosp_transaction:
803                     hosp_transaction[hosp_code] = []
804                 hosp_transaction[hosp_code].append(line.strip())
805
806         if transactions_found:
807             print(f"Transaction report for items distributed in {month}:")
808             for hosp_code, transactions in hosp_transaction.items():
809                 print(f"\nTransactions for hospital {hosp_code}:")
810                 for transaction in transactions:
811                     print(transaction)
812         else:
813             print(f"There are no transactions for items distributed in {month}.")
814
815         print(" ")
816
817         receive_content = readlines_file(receive_path)
818
819         # Transaction report for supply received
820         receive_found = False
821         receive = {}
822
823         for line in receive_content:
824             # If month found
825             if month in line:
826                 receive_found = True
827                 parts = line.strip().split(',')
828                 supp_code = parts[2]
829                 if supp_code not in receive:
830                     receive[supp_code] = []
831                 receive[supp_code].append(line.strip())
832
833         if receive_found:
834             print(f"Transaction report for items received from suppliers in {month}:")
835             for supp_code, transactions in receive.items():
836                 print(f"\nTransactions for items received from {supp_code}:")
837                 for transaction in transactions:
838                     print(transaction)
839         else:
840             print(f"There are no transactions for items received in {month}.")
841
842         return

```

The function `overall_transaction()` is defined. The collection `valid month` contains valid month names and is used to validate user input. The function then enters a loop that prompts the user to enter a month and ensures that the user input is valid by checking `valid month`.

If the month is valid, the function reads the distribution file contents using `readlines_file(dist_path)` and initializes two variables: `transactions_found` as `False` and `hosp_transactions` as empty dictionary.

The program then sets `transactions_found` to `True` if it finds the specified month in a row and extracts the hospital code. If the hospital code does not exist in `hosp_transaction`, it is added as a key with an empty list as its value. Each matching transaction row is appended to the corresponding list of hospital codes.

If a transaction is found, the program prints a report of items distributed in the specified month, grouped by hospital code. If no transaction is found, a message is displayed indicating that there were no transactions for that month.

Next, the function reads the received supplies file contents using `readlines_file(receive_path)` and initializes two variables: `receive_found` as `False` and `receives` as empty dictionary.

The program then sets `receive_found` to `True` if the specified month is found in a line and extracts the supplier code. If the supplier code does not exist in `receive`, it is added as a key with an empty list as its value. Each matching transaction line is appended to the corresponding list of supplier codes.

If the received supplies are found, the program prints a report of items received for the specified month, grouped by supplier code. If the supplies received are not found, a message is displayed indicating that there were no transactions for that month. After generating and printing the report, the function exits

1.19 Home Function & Code

```
# Home Code
def home():
    print("Welcome to the menu!")

    # Call functions
    supplier_file(supp_path)
    create_ppe_file(ppe_path, supp_path)

    while True:

        print("""What would you like to do?
        '1' to Create Hospital File
        '2' to Update Inventory
        '3' to Update Supplier Details
        '4' to Update Hospital Details
        '5' to Track Inventory Items
        '6' to Search Distribution List
        '7' to Print Report
        '8' to Log Out""")

        home_input = input("What would you like to do?")

        # Create hosp file
        if home_input == "1":
            hospital_file(hosp_path)
            # break

        # Update inventory
        elif home_input == "2":
            while True:
                # Add or Distribute
                invent_input = input("Would you like to add inventory or distribute inventory? [1 to Add Inventory, 2 to Distribute Inventory]\n> ")

                # Add inventory
                if invent_input == "1":
                    add_invent(ppe_path, receive_path)
                    break

                # Distribute inventory
                elif invent_input == "2":
                    distribute_invent(hosp_path, dist_path, ppe_path)
                    break

                else:
                    print("This is not a valid answer. Please try again")

            # Update supplier details
            elif home_input == "3":
                update_supp_details(supp_path)

            # Update Hospital Details
            elif home_input == "4":
                update_hosp_details(hosp_path)

            # Track Items
            elif home_input == "5":
                track_items()

            # Search Distribution List
            elif home_input == "6":
                search_distribution(ppe_path)

            elif home_input == "7":
```

```

while True:
    print("""What would you like to do?
    '1' to Print the List of Suppliers with their PPE equipments supplied
    '2' to Print the List of Hospitals with quantity of distribution items
    '3' to Print the Overall Transaction Report for a selected month
    '4' to Quit Print Report\n""")

    user_input = input("Would you like to print the list of suppliers with their PPE equipments supplied or the "
                       "list of hospitals with quantity of distribution items or overall transaction report for a selected month?")

    # Print List of Suppliers with PPE supplied
    if user_input == '1':
        supp_with_ppe(supp_path, ppe_path)
        break

    # Print List of Hospitals with quantity of distribution items
    elif user_input == '2':
        distribution_hospital()
        break

    # Print Overall Transaction report
    elif user_input == '3':
        overall_transaction(dist_path, receive_path)
        break

    elif user_input == '4':
        break

    else:
        print("Please enter correct input.")

# Log Out
elif home_input == "8":
    print("Logging out...")
    break

else:
    print("Invalid input. Please try again.")
    continue

if __name__ == '__main__':
    main()

```

Lastly, the “home()” function is defined. The functions `supplier_path` and `create_ppe_path` is called to initialize the text files beforehand as both functions are crucial to the execution of the code. After that, the user is prompted to choose among 8 choices, including creating hospital file, updating inventory, updating supplier details, updating hospital details, track inventory items, search distribution list, print reports or log out.

If the user chooses to create hospital file, “`hospital_file(hospital_path)`” function is called.

If the user chooses to update inventory, they will be prompted again to add to the inventory or distribute inventory. If the user wants to add inventory, “`add_invent(ppe_path, receive_path)`” function is called. If the user wants to distribute inventory, “`distribute_invent(hosp_path, dist_path, ppe_path)`” is called.

If the user chooses to update supplier details, the function “`update_supp_details(supp_path)`” is called.

If the user chooses to update hospital details, the function “update_hosp_details(hosp_path)” is called.

If the user chooses to track inventory items, the function “track_items()” is called.

If the user chooses to search distribution list, the function “search_distribution(ppe_path)” is called.

If the user chooses to print reports, they will be prompted further to select which report to print. If the user chooses to print the list of suppliers with their PPE equipment supplied, “supp_with_ppe(supp_path, ppe_path)” function is called. If the user chooses to print the list of hospitals with quantity of distribution items, “distribution_hospital()” function is called. If the user chooses to print the overall transaction report for a selected month, “overall_transaction(dist_path, receive_path) function is called. If the user chooses to quit, the loop will be exited.

If the user chooses to log out, the function ends.

If the user chooses none of the options above, an error will occur and the user will be asked to reinput their choice.

Finally, the last two lines of the code represents a built-in Python variable “__name__” which will be set to __main__. This checks whether the script is run directly or being imported in which the output will be “True” or “False” respectively. If the output is “True”, the function “main()” will run.

2.0 Sample Input/Output with Explanation

2.1 Logging In

```
Would you like to sign up or log in?  
1. Sign up  
2. Log in  
3. Quit Program  
1
```

```
Please enter a usernamejaydenfoo123  
Please enter your passwordjayden123  
Your username is jaydenfoo123  
Your password is jayden123  
Sign up successfully  
You have logged in  
Welcome to the home page  
Welcome to the menu!
```

At the start of entering our inventory management system, you have an option to sign up or log in.

In this example, '1' is the input, which means I would like to sign up. Therefore, my username is "jaydenfoo123" and password is "jaydenfoo123". After signing up is successful, the system will automatically log you in and you be directed to the home page.

2.2 Creating Supplier File

```
supplier.txt does not exist. Creating a new file.  
Example: S01, Supplier Name, 1 Jalan XXX Taman XXX, 012-3456789  
(The user input should be exactly the same as the expected format)  
Noted: The address should not include any ','  
Please enter supplier details or type 'done' to finish:
```

```
Please enter supplier details or type 'done' to finish: S01, Elvan Sea, Jalan PJU 5/6 Taman Genni, 013-4430987  
['S01', 'Elvan Sea', 'Jalan PJU 5/6 Taman Genni', '013-4430987']  
Data written to file.  
Please enter supplier details or type 'done' to finish: done  
supplier.txt has been created with your input.
```

For the first time user or first logged in, at least one supplier.txt must be created, where maximum 3 to 4 suppliers can be added. To input supplier.txt, the expected format must be followed as given by the example provided. After the first successful input, you can proceed to input another 3 more. If you are done, you can input “done”. Then, an output will be printed indicating that supplier.txt has been created successfully.

2.3 PPE File Creation

ppe.txt does not exist. Creating a new file.

Example: HC, Head Cover, 100, S01, January 27 2024

(The user input should be exactly the same as the expected format)

```
Enter text to write into this file or 'done' to finish:HC, Head Cover, 100, S01, January 27 2024
['HC', 'Head Cover', '100', 'S01', 'January 27 2024']
Data written to file.
Enter text to write into this file or 'done' to finish:done
ppe.txt has been created with your input
```

Similarly, upon a user first time logged in, at least one ppe.txt must be created. To input supplier.txt, the expected format must be followed as given by the example provided. If you are done, you can input “done” and an output will be printed indicating that ppe.txt have been created successfully. After that, you will be directed to the homepage for further actions to be performed.

2.4 Home Page - Option 1

```
What would you like to do?
    '1' to Create Hospital File
    '2' to Update Inventory
    '3' to Update Supplier Details
    '4' to Update Hospital Details
    '5' to Track Inventory Items
    '6' to Search Distribution List
    '7' to Print Report
    '8' to Log Out
What would you like to do?1
```

```
Would you like to create hospital.txt file?
    '1' for Yes
    '2' for No1
Example: H01, Hospital XXX
Please enter hospital details or type 'done' to finish: H02, Hospital Genni
['H02', 'Hospital Genni']
Data written to file.
Please enter hospital details or type 'done' to finish: done
hospital.txt has been created with your input.
```

There are 8 options that can be performed on the homepage. When the user is prompt “What would you like to do”, user will input any number from 1-8, which as above, I input “1”. Option “1” is to create hospital.txt file, where user is required to enter the respective hospital details based on the given format. When user is finished, “done” must be the input and an output will be printed indicating that hospital.txt have been created successfully.

2.5 Home Page - Option 2

```
What would you like to do?
    '1' to Create Hospital File
    '2' to Update Inventory
    '3' to Update Supplier Details
    '4' to Update Hospital Details
    '5' to Track Inventory Items
    '6' to Search Distribution List
    '7' to Print Report
    '8' to Log Out
What would you like to do?2

Would you like to add to the inventory or distribute inventory? [1 to Add Inventory, 2 to Distribute Inventory]
> 1
What item do you want to add? (Please enter item code): HC
How many items to add?: 100
Inventory updated successfully.
```

If the user selects option '2' to update inventory, user will have the option to add inventory or distribute inventory, each represented by '1' and '2' respectively. In this example, I input option '1' to add inventory, where I am required to enter item code and input quantity of items to add. Then, an output indicating that inventory is updated successfully will be printed.

2.6 Home Page - Option 3

```
What would you like to do?
    '1' to Create Hospital File
    '2' to Update Inventory
    '3' to Update Supplier Details
    '4' to Update Hospital Details
    '5' to Track Inventory Items
    '6' to Search Distribution List
    '7' to Print Report
    '8' to Log Out
What would you like to do?3
```

```
Please enter the supplier code: S01
Which detail do you want to edit? [Name, Address, Contact]: Name
Please enter the new name: Elvan Sea
Supplier name updated successfully.
```

If the user selects option '3' to update supplier details, user will first need to enter supplier code. From here, users will have to input what they would like to edit, either name, address or contact. In the example above, names have been selected to be edited. Then, the new name will be the input and an output indicating that supplier name has been updated successfully will be printed.

2.7 Home Page - Option 4

```
What would you like to do?
    '1' to Create Hospital File
    '2' to Update Inventory
    '3' to Update Supplier Details
    '4' to Update Hospital Details
    '5' to Track Inventory Items
    '6' to Search Distribution List
    '7' to Print Report
    '8' to Log Out
What would you like to do?4
```

```
Please enter the hospital code: H01
Which detail do you want to edit? [Name]Name
Please enter the new name: Genni Kee
Hospital name updated successfully.
```

When a user input option '4', they can update hospital details. Users will first need to enter hospital code. From here, users will have to input the hospital code and detail that the user likes to edit. Then, the new name will be the input and an output indicating that the hospital name has been updated successfully will be printed.

2.8 Home Page - Option 5

```
What would you like to do?
    '1' to Create Hospital File
    '2' to Update Inventory
    '3' to Update Supplier Details
    '4' to Update Hospital Details
    '5' to Track Inventory Items
    '6' to Search Distribution List
    '7' to Print Report
    '8' to Log Out
What would you like to do?5
```

Do you want to track available quantity of all items or track all items which has less than 25 stock? [1 or 2]1

Item Code: FS; Quantity: 100

Item Code: GL; Quantity: 100

Item Code: GW; Quantity: 100

Item Code: HC; Quantity: 100

Item Code: MS; Quantity: 100

Item Code: SC; Quantity: 100

Option “5” can be initiated to track inventory items. When user input ‘5’, they need to choose to either track all available quantity or track inventory that have less than 25 in stock, both with “1” and “2” assigned respectively.

2.9 Home Page - Option 8

```
What would you like to do?
    '1' to Create Hospital File
    '2' to Update Inventory
    '3' to Update Supplier Details
    '4' to Update Hospital Details
    '5' to Track Inventory Items
    '6' to Search Distribution List
    '7' to Print Report
    '8' to Log Out
What would you like to do?8
```

```
Logging out...
Would you like to sign up or log in?
1. Sign up
2. Log in
3. Quit Program
```

To quit or log out, user must input “8”. Then, the system will print an output “logging out”. By then, the user will be redirected to the signup/login page where they have the option to log in again. At this page, they can input ‘3’ to quit the program completely.

3.0 Screenshots of Text Files

PPE File (ppe.txt)

```
1 HC, Head Cover, 100, S01, January 27 2024
2 FS, Face Shield, 100, S03, January 27 2024
3 MS, Mask, 100, S02, January 27 2024
4 GL, Gloves, 100, S02, January 27 2024
5 GW, Gown, 100, S01, January 27 2024
6 SC, Shoe Covers, 100, S03, January 27 2024
```

User Info File (user_info.txt)

```
1 jayden, jayden123
```

Supplier Info File (supplier.txt)

```
1 S01, Jayden, 1 Jalan Tun Husein Onn Taman Gembira, 011-1111111
2 S02, Elvan, 2 Jalan Ampang Taman Ampang, 012-2223456
3 S03, Aurora, 3 Jalan Cheras Taman Federal, 014-83765931
4 S04, Genni, 4 Jalan XXX Taman XXX, 011-34545623
```

Hospital Info File (hospital.txt)

```
1 H01, Hospital Subang Jaya
2 H02, Hospital Pantai
3 H03, Hospitald Columbia
4 H04, Hospital Ampang
```


Distribution File (distribution.txt)

1	HC, 1, H01, <u>June 07 2024</u>
2	HC, 2, H02, <u>July 07 2024</u>
3	HC, 1, H01, <u>July 07 2024</u>
4	HC, 10, H01, <u>July 07 2024</u>
5	HC, 1, H02, <u>July 07 2024</u>
6	HC, 1, H02, <u>July 07 2024</u>
7	HC, 10, H02, <u>July 07 2024</u>
8	HC, 10, H01, <u>July 07 2024</u>
9	HC, 10, H01, <u>July 07 2024</u>
10	HC, 10, H02, <u>July 07 2024</u>
11	HC, 10, H02, <u>July 08 2024</u>
12	HC, 9, H02, <u>July 08 2024</u>
13	HC, 3, H02, <u>July 08 2024</u>
14	HC, 89, H02, <u>July 15 2024</u>
15	MS, 20, H03, <u>July 21 2024</u>
16	MS, 12, H03, <u>July 21 2024</u>
17	MS, 1, H01, <u>July 21 2024</u>
18	MS, 1, H02, <u>July 21 2024</u>
19	MS, 1, H01, <u>July 21 2024</u>
20	MS, 5, H02, <u>July 21 2024</u>
21	

Supply Received File (receive_supply.txt)

1	HC, 8, S03, <u>July 16 2024</u>
2	HC, 8, S03, <u>July 16 2024</u>
3	HC, 90, S03, <u>July 16 2024</u>
4	HC, 2, S03, <u>July 16 2024</u>
5	GL, 9, S03, <u>July 16 2024</u>
6	MS, 2, S02, <u>July 21 2024</u>
7	MS, 1, S02, <u>July 21 2024</u>
8	MS, 1, S02, <u>July 21 2024</u>
9	MS, 3, S02, <u>July 21 2024</u>
10	MS, 7, S02, <u>July 21 2024</u>
11	MS, 1, S02, <u>July 21 2024</u>
12	MS, 3, S02, <u>July 21 2024</u>
13	MS, 1, S02, <u>July 21 2024</u>

4.0 Conclusion

In conclusion, the Personal Protective Equipment (PPE) Inventory Management System has been successfully developed. This system has met the requirements of the Ministry of Health and effectively manages the inventory of PPE items received from multiple suppliers and distributed to various hospitals. The main features of the system include:

1. Secure Login Functionality

The system ensures that only authorized inventory controllers can access and manage inventory, and they can make up to three login attempts. In this way, the system's security can be increased.

2. Initial Inventory Creation

During the first execution, the system initializes the inventory with a predefined number of PPE items. The user is prompted to enter details and store the information in ppe.txt.

3. Inventory Update

This system allows for seamless inventory updates, including receiving new supplies and distributing PPE items to hospitals. Before distributing PPE, it verifies that there is sufficient stock available.

4. Inventory Tracking

The program can print the total available quantity of all items sorted by item code. Additionally, it identifies items with fewer than 25 boxes in stock for efficient inventory management.

5. Search Functionality

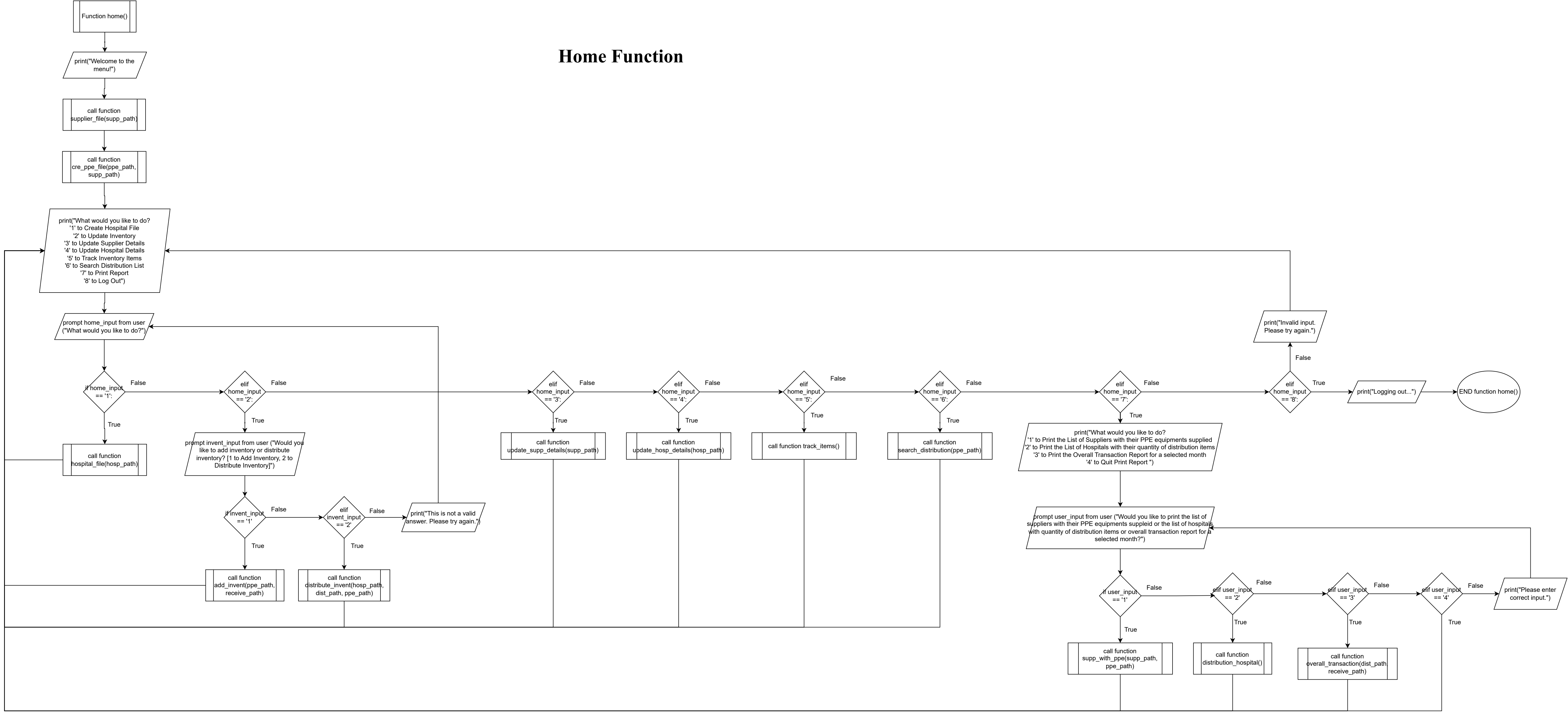
The system provides details about the hospital code and the number of distributions. As a result, users can search for the distribution history of a specific product by item code.

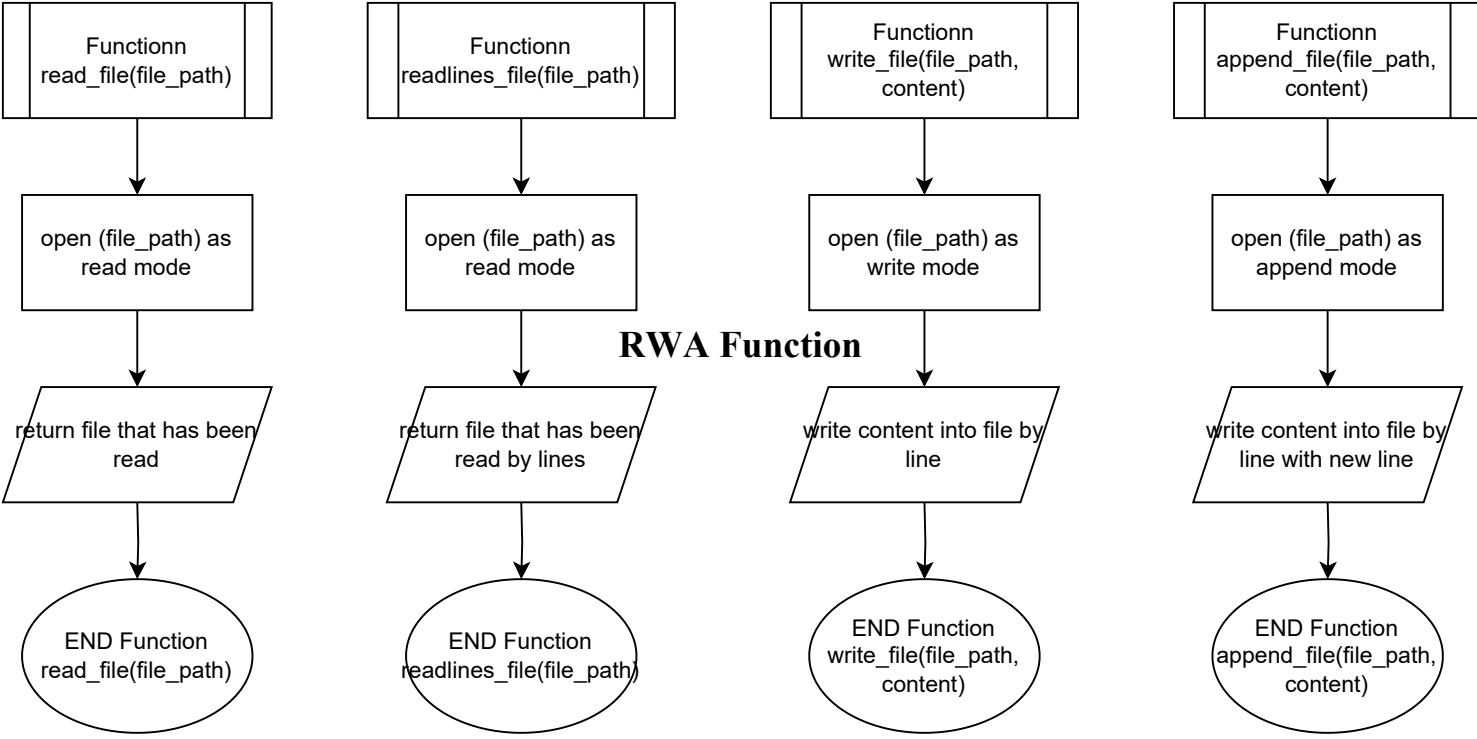
6. Report Generation

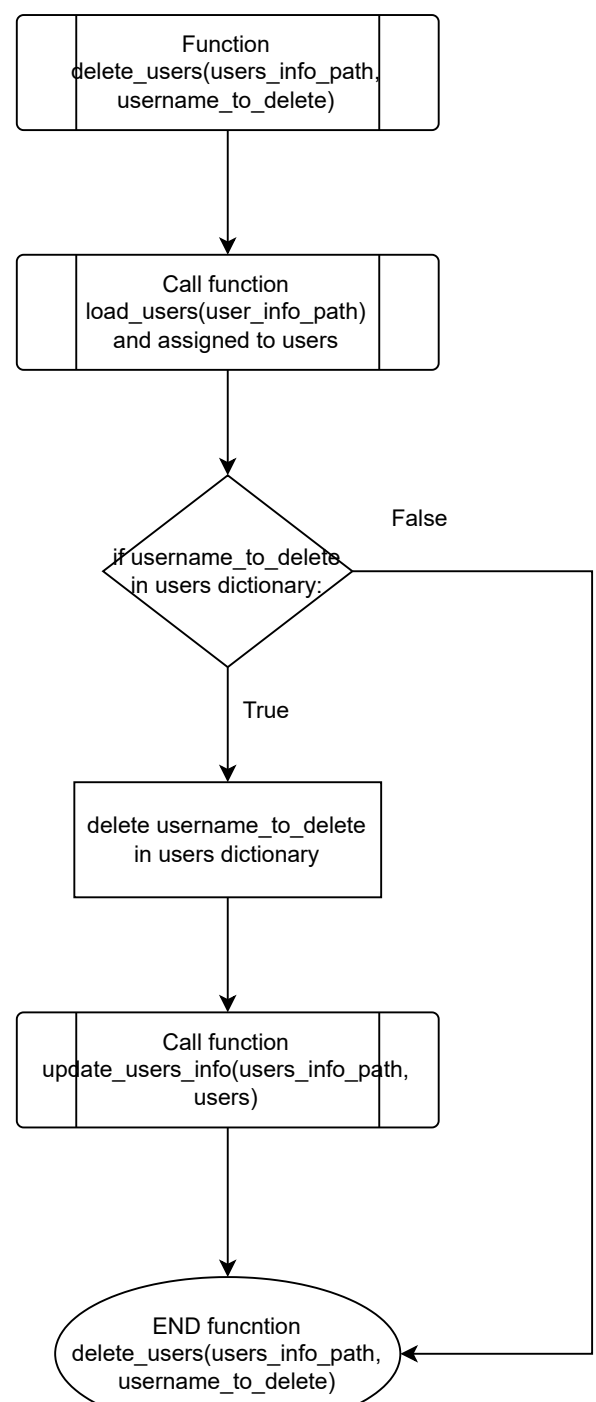
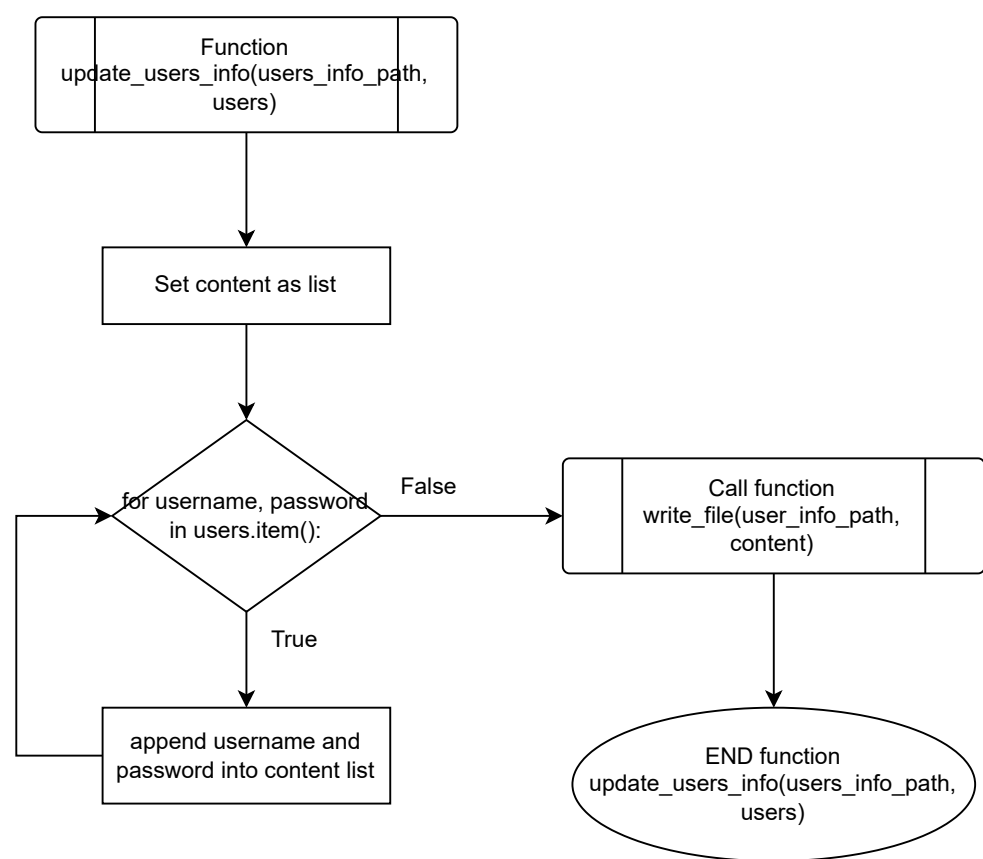
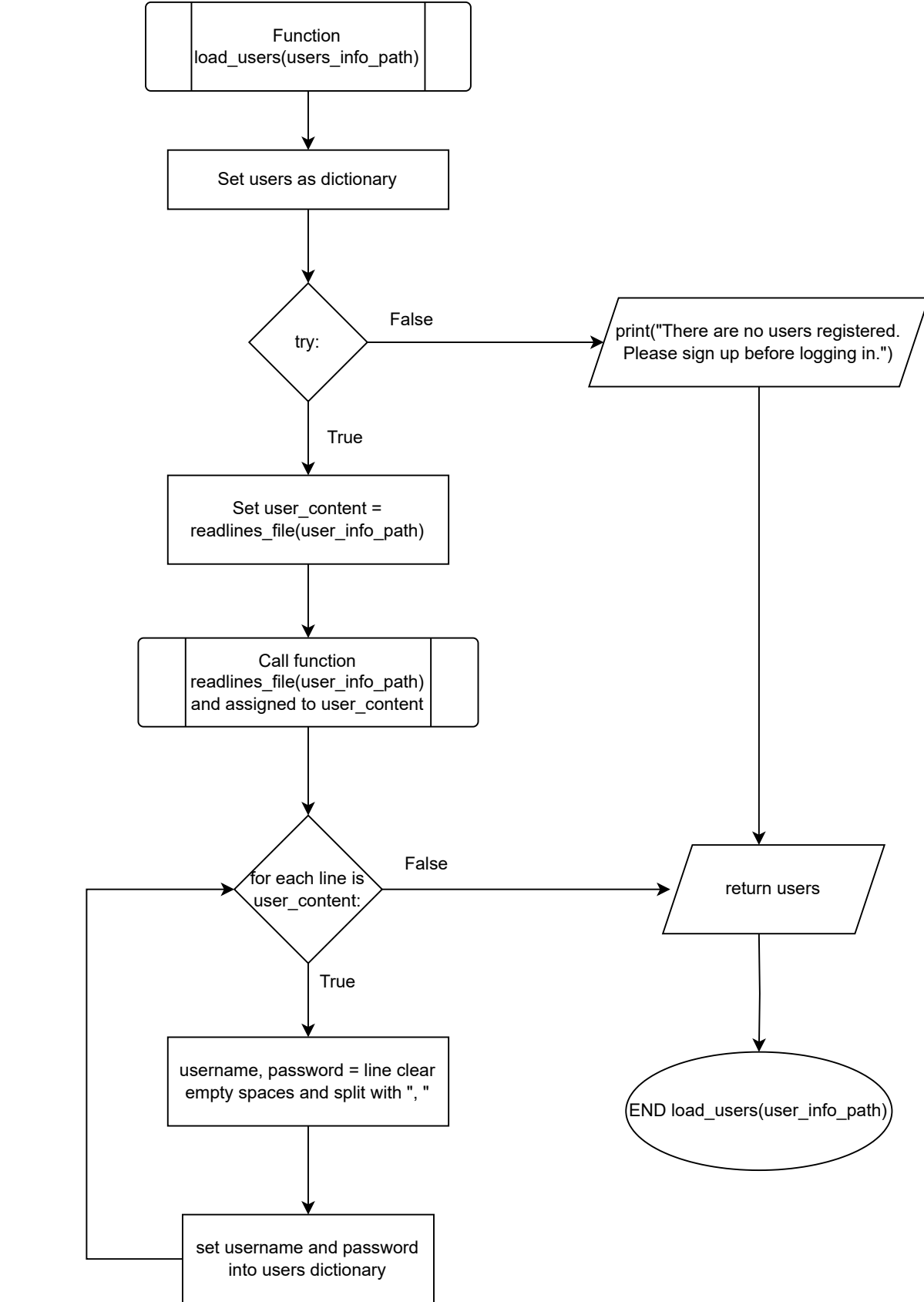
A variety of reports are generated, including a list of suppliers with their items, hospitals with their quantity distribution of items that received from suppliers, and overall transaction report for the selected month.

Throughout the development process, we adhere to good programming practices such as modular design, proper use of functions, and comprehensive validation. We have tested the system and made sure that the system can handle both valid and invalid inputs, increasing the reliability of the system.

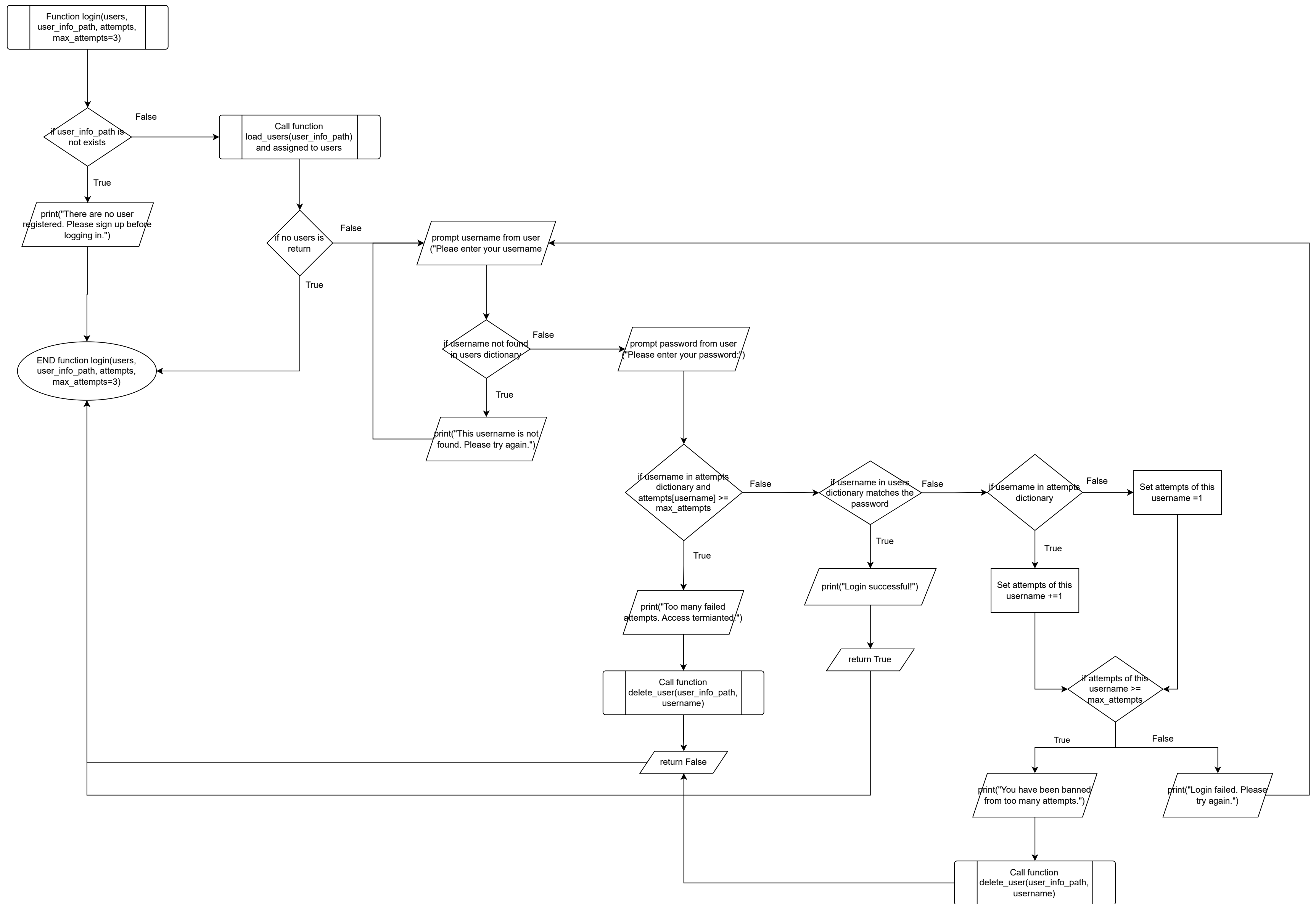
Home Function



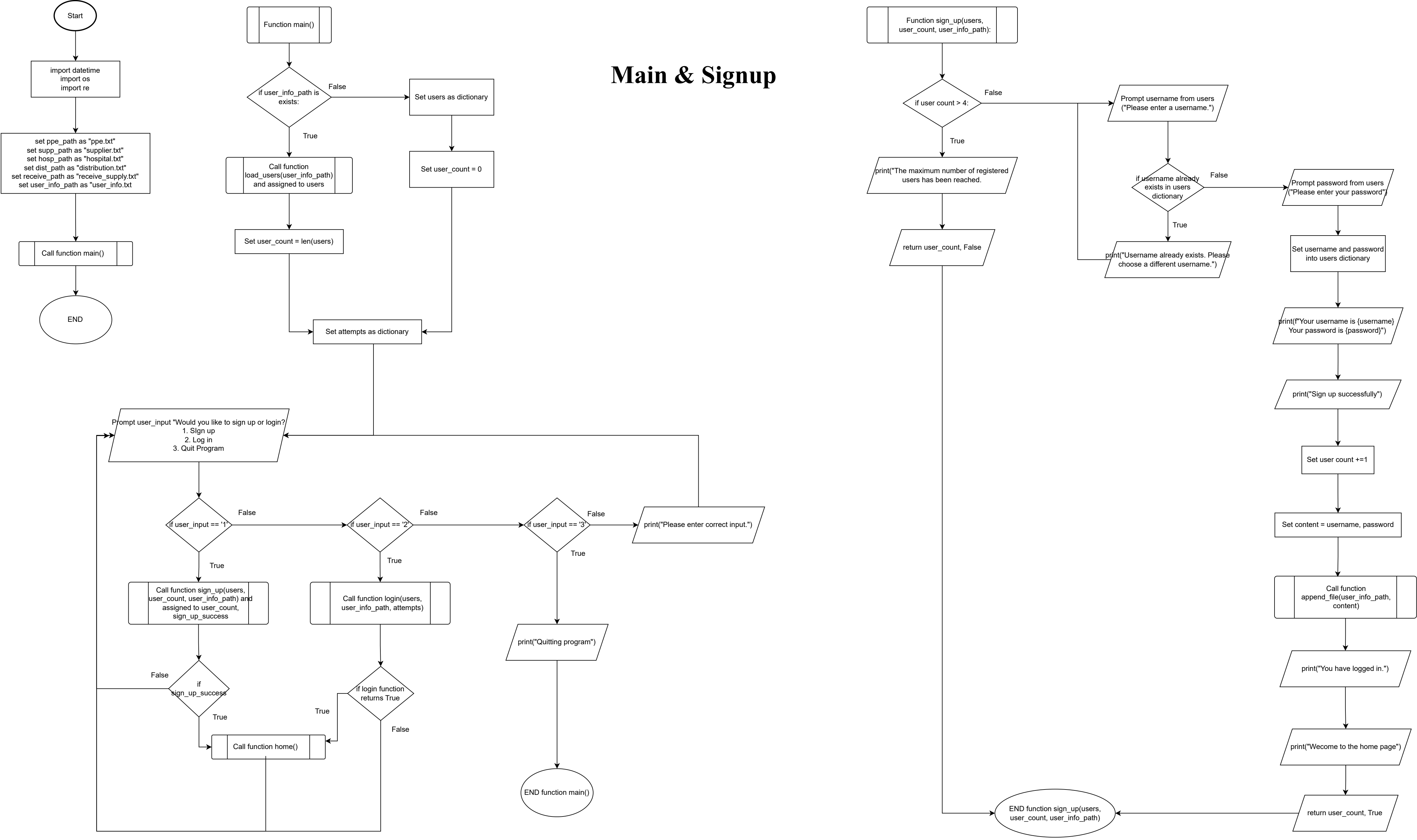




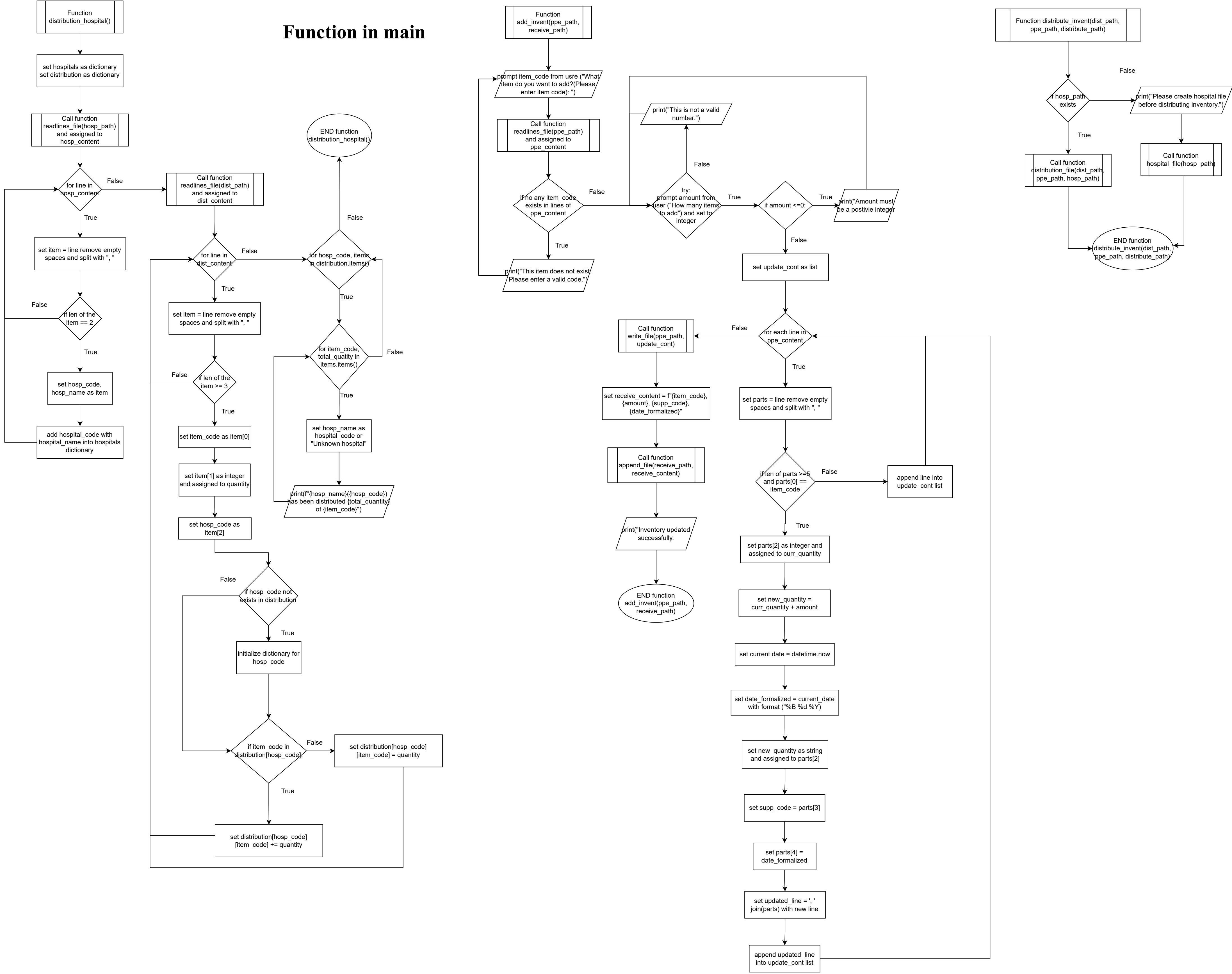
Login Function

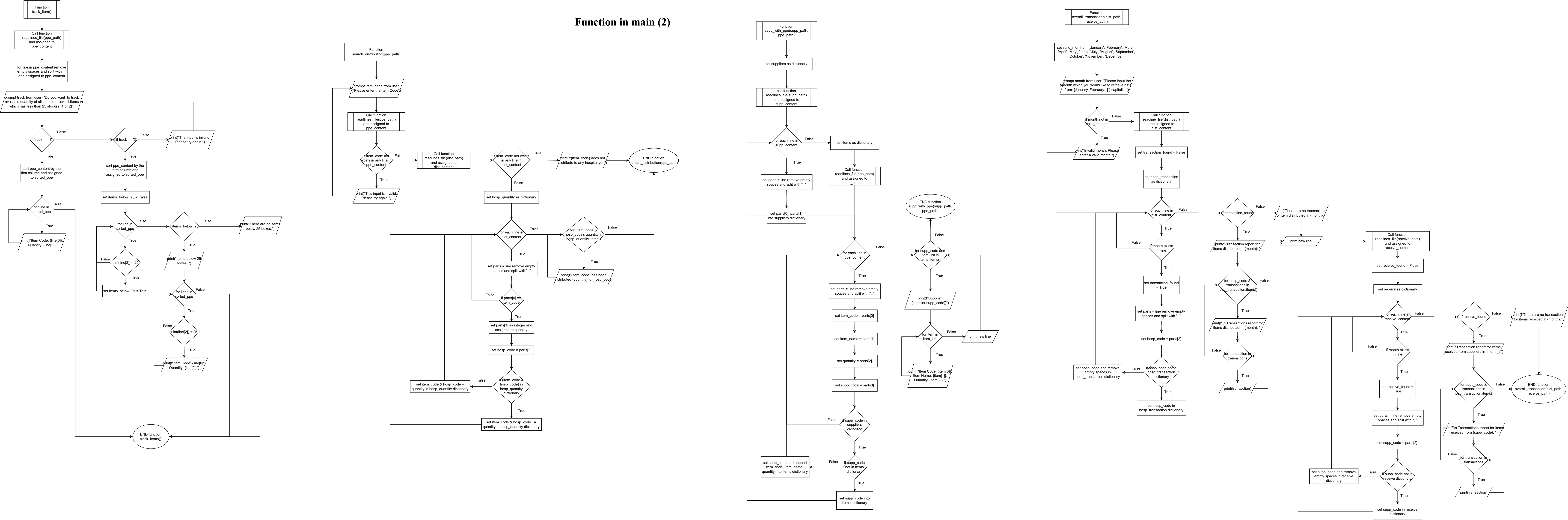


Main & Signup



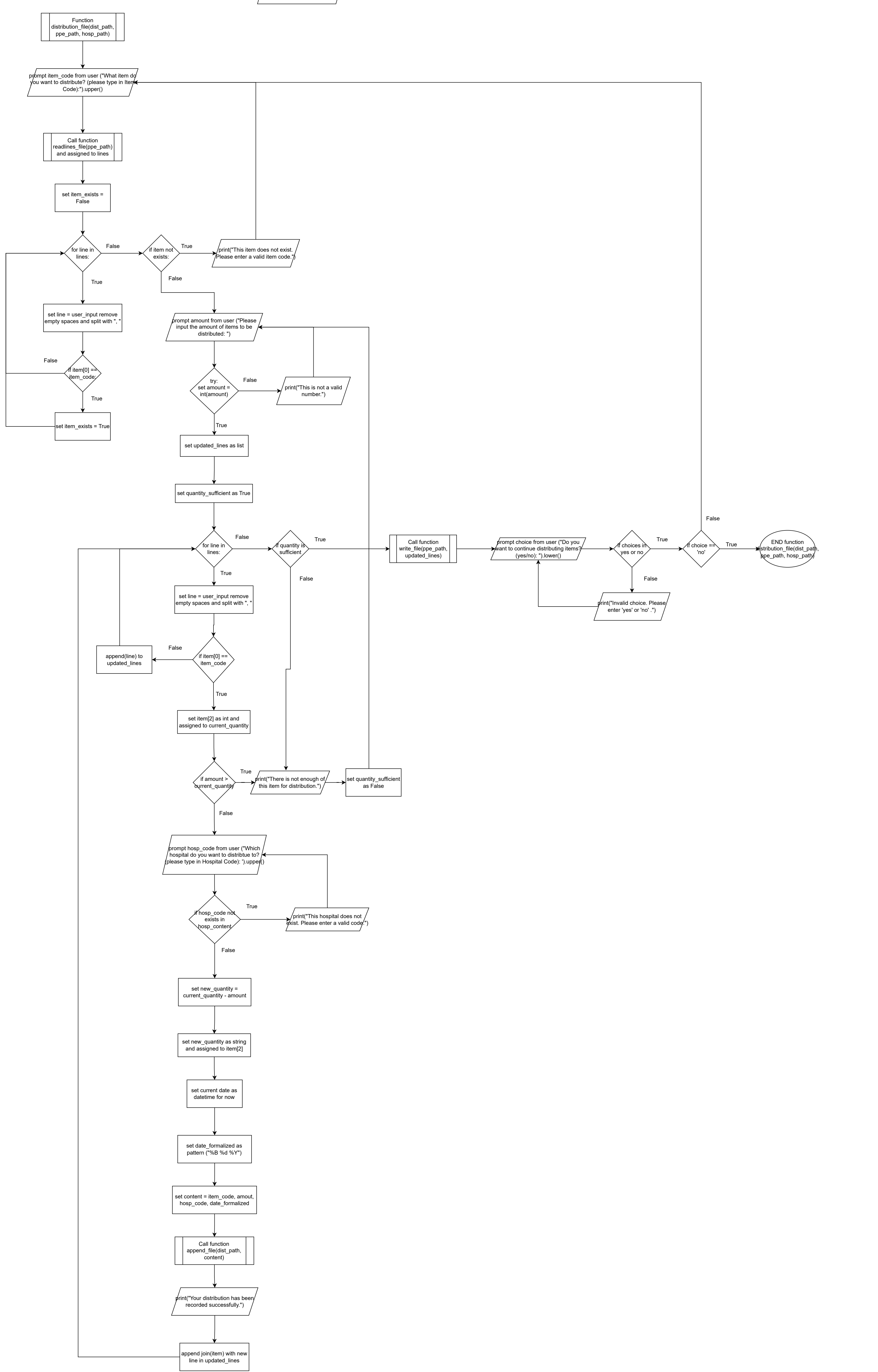
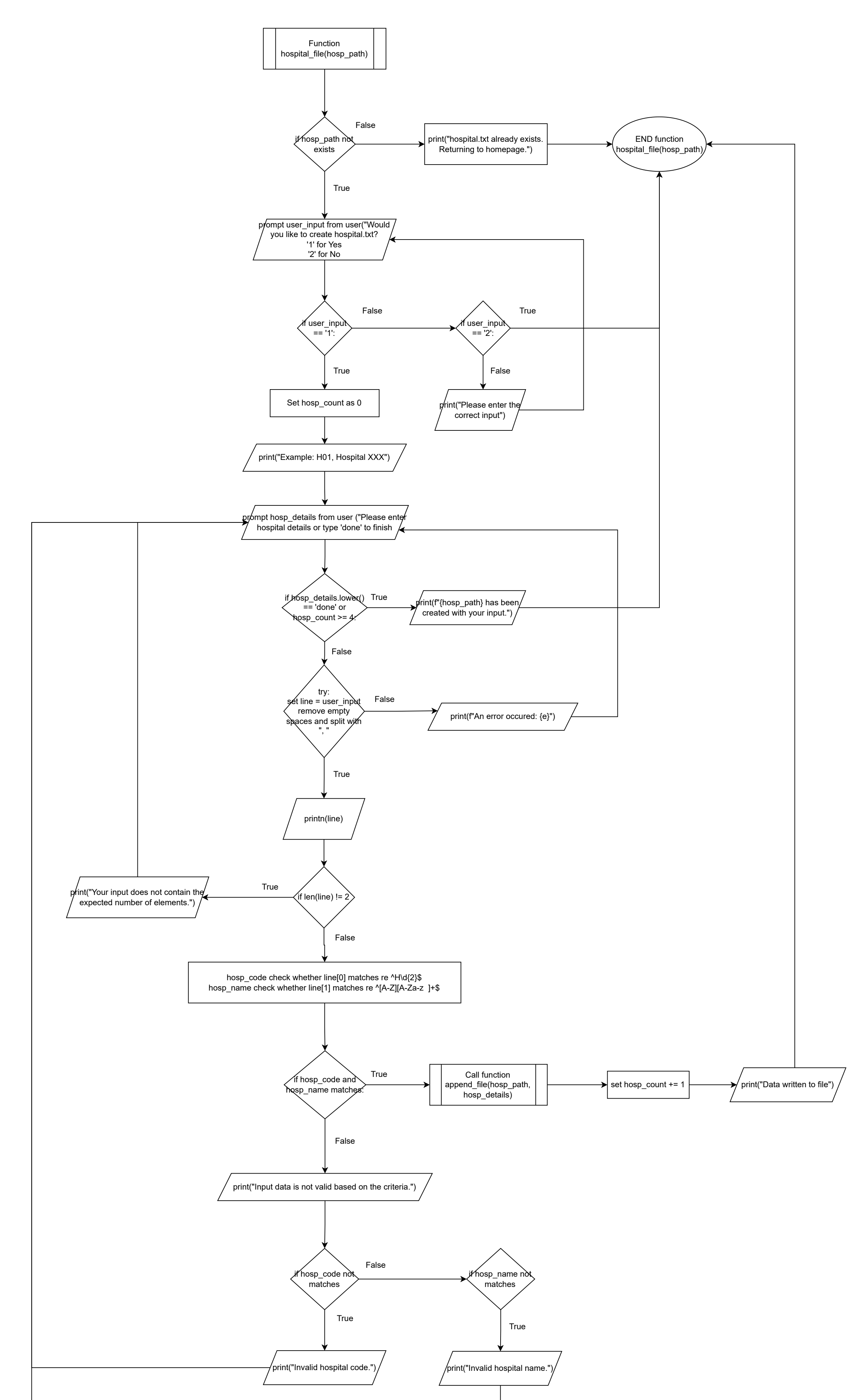
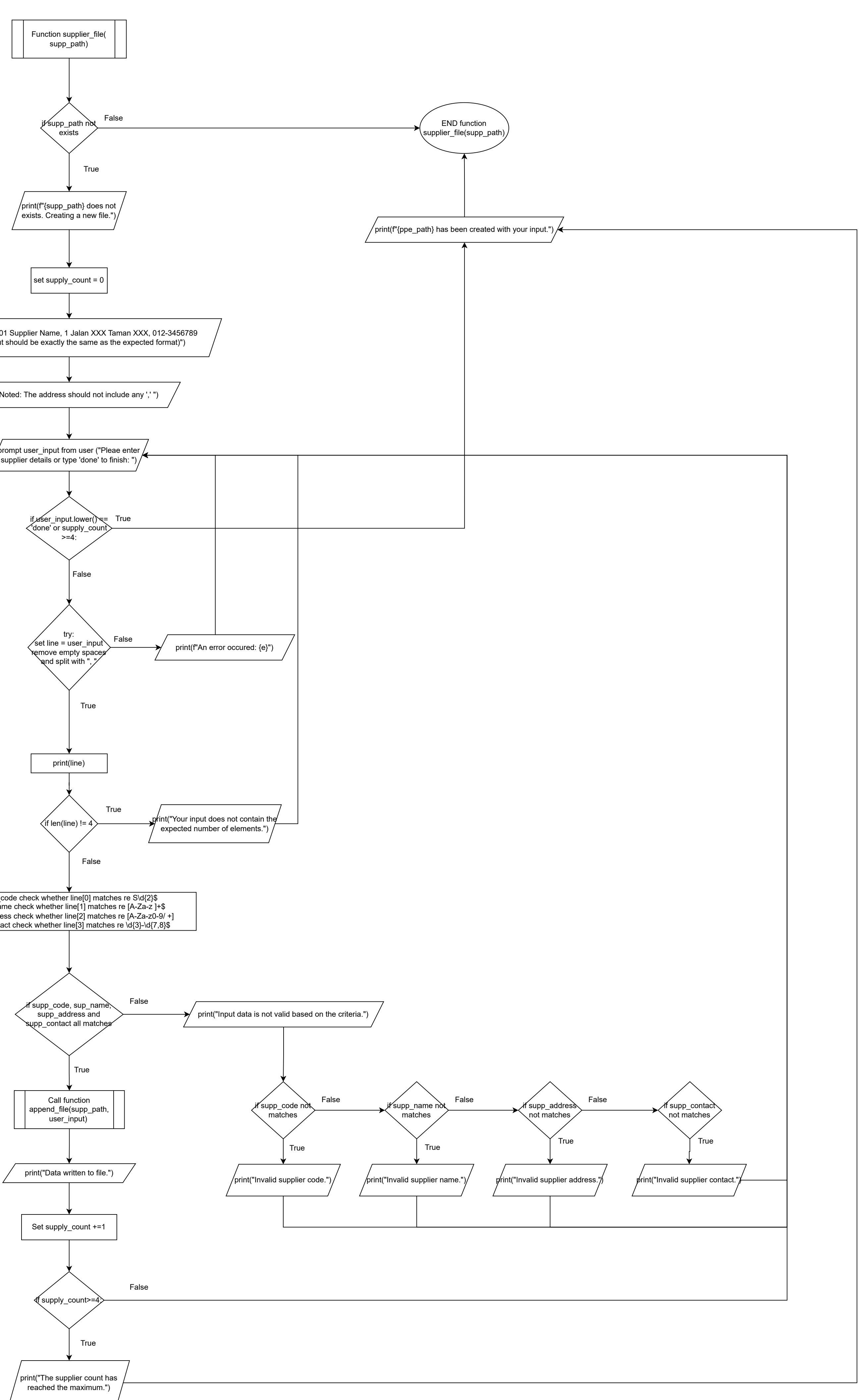
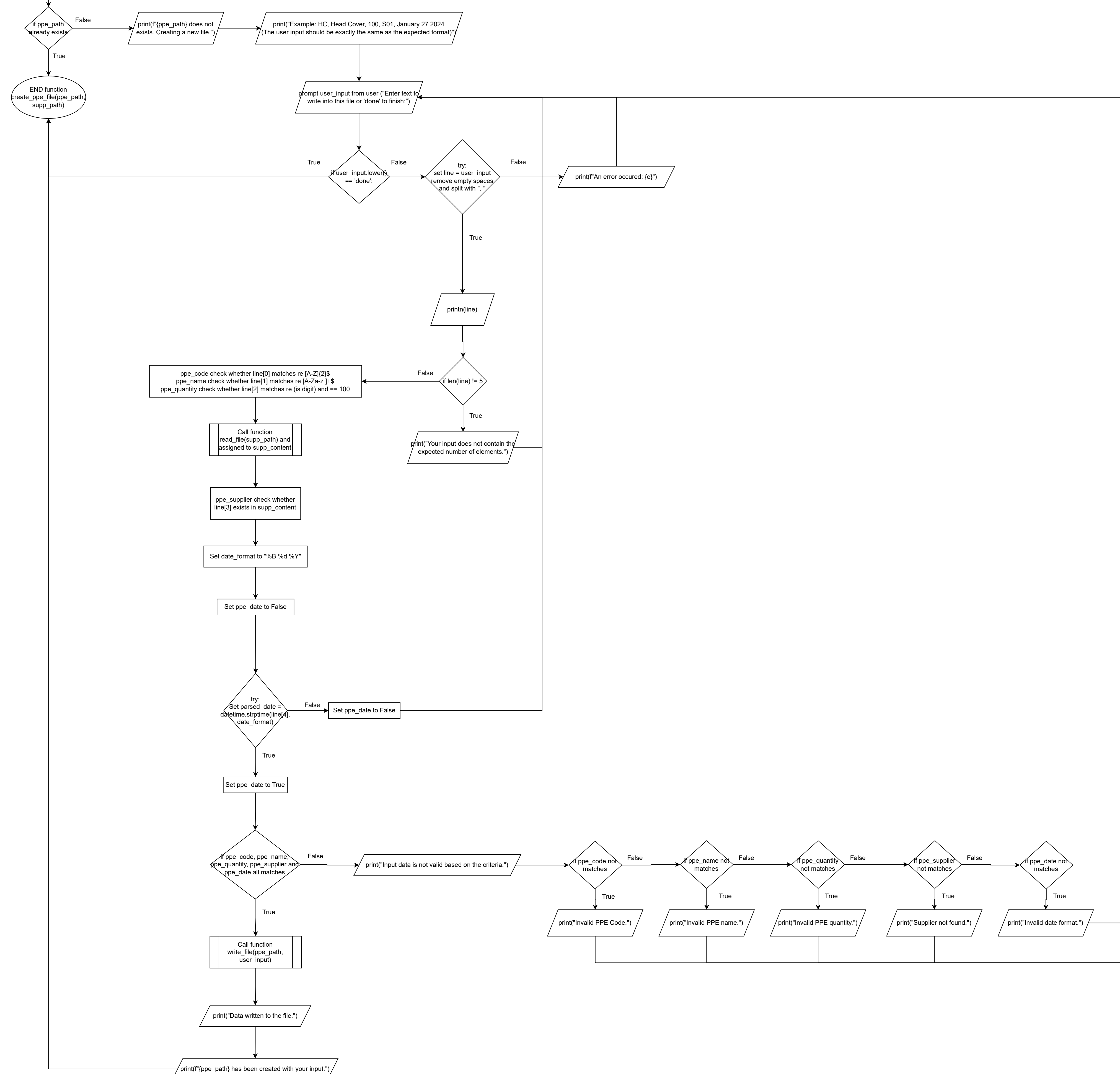
Function in main





Function
create_ppe_file(ppe_path,
supp_path)

Text File Function



Update Function

